

MLP를 Top-Down으로 이해하기

비선형 분류 문제에서 시작해, 한 개의 숫자 계산 → 역전파 → 실제 데이터 학습까지

이 자료는 제공된 다음 실습 자료의 흐름과 용어를 기반으로 재구성한 초심자용 학습서임.

- 간단한 Perceptron 구현: `make_moons`, `Tensor`, `autograd`, `nn.Module`, `DataLoader`, `optimizer`, `binary cross entropy`, 결정 경계
- MLP 구현 실습: `load_digits`, 3-layer MLP, `ReLU`, `Dropout`, `CrossEntropyLoss`, `Adam`, `train()/eval()`, Early Stopping
- 신경망과 MLP 쉬운 자료: 선형분류기 → XOR/비선형 문제 → 함수 중첩과 활성화 함수 → PyTorch → MLP

자료에서 직접 설명되지 않은 세부 수식 유도과 시각화는 추가 해설로 표시함.

학습 전략: 먼저 “왜 MLP가 필요한가?”를 본 뒤 내부로 내려감

[최종 문제]

곡선 모양으로 섞인 두 클래스를 어떻게 분류할까?



[비교]

Perceptron은 직선 하나만 만들 수 있음



[해결 아이디어]

여러 Linear + 비선형 활성화 함수



[MLP]

Linear → ReLU → Linear → ReLU → Linear



[한 샘플 내부 계산]

가중합 → 활성화 → logit → 확률 → loss



[학습]

loss.backward() → gradient → optimizer.step()



[배치/데이터셋]

DataLoader → 반복 학습



[실제 문제]

8×8 손글씨 숫자 0~9 분류

최종 학습 목표

1. Perceptron과 MLP의 구조 차이를 **결정 경계** 관점에서 설명 가능
2. **Linear** 한 층에서 계산되는 **$Wx+b$** 를 숫자로 직접 계산 가능
3. ReLU가 왜 필요한지 **비선형성** 관점에서 설명 가능
4. 이진 분류의 **logit → sigmoid → BCE loss** 흐름 이해
5. **loss.backward()** 가 어떤 미분값을 계산하는지 단일 샘플에서 추적 가능
6. **DataLoader → forward → loss → backward → optimizer.step()** 학습 루프 이해
7. **load_digits** 다중 클래스 문제에서 **CrossEntropyLoss** 와 **argmax** 사용 이유 이해

```
# 0. 환경 준비
import math
import random
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_moons, load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

SEED = 42
```

```

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

pd.set_option("display.precision", 4)

print("PyTorch:", torch.__version__)
print("Device :", DEVICE)

```

PyTorch: 2.10.0+cpu
Device : cpu

한글 그래프 폰트 자동 설정

Matplotlib/Seaborn에서 한글이 ☐ 또는 깨진 문자로 출력되는 문제를 방지함.

환경별로 다음 글꼴을 우선 탐색함.

Malgun Gothic → AppleGothic → Noto Sans CJK → NanumGothic → NanumBarunGothic

사용 가능한 한글 폰트를 자동 선택하고, Matplotlib의 음수 기호(−) 깨짐도 함께 방지함.

별도의 폰트 파일을 노트북에 포함하지 않음. 실행 환경에 설치된 폰트 중 사용 가능한 항목을 선택하는 방식임.

```

# 한글 폰트 자동 설정: Matplotlib + Seaborn
import os
from matplotlib import font_manager, rcParams

def setup_korean_font(verbose=True):
    """
    현재 실행 환경에서 사용 가능한 한글 폰트를 자동 탐색하여
    Matplotlib/Seaborn의 기본 폰트로 설정함.
    """

```

```

preferred_names = [
    "Malgun Gothic",      # Windows
    "AppleGothic",        # macOS
    "Apple SD Gothic Neo", # macOS
    "Noto Sans CJK KR",    # Linux / Colab 계열
    "Noto Sans KR",
    "NanumGothic",
    "NanumBarunGothic",
    "UnDotum",
    "Baekmuk Gulim",
]

# Matplotlib이 인식하는 폰트 이름 목록
installed = {f.name: f.fname for f in font_manager.fontManager.ttflist}

selected_name = None
selected_path = None

# 1) 정확한 family name 우선
for name in preferred_names:
    if name in installed:
        selected_name = name
        selected_path = installed[name]
        break

# 2) family name 탐색 실패 시 시스템 폰트 경로명으로 보조 탐색
if selected_name is None:
    keywords = [
        "malgun", "applegothic", "applesdgothic",
        "notosansCJK", "notosansKR",
        "nanumgothic", "nanumbarungothic",
        "undotum", "gulim"
    ]
    system_fonts = (
        font_manager.findSystemFonts(fontext="ttf")
        + font_manager.findSystemFonts(fontext="otf")
    )

    for path in system_fonts:
        norm = path.lower().replace(" ", "")
        if any(key in norm for key in keywords):
            try:
                prop = font_manager.FontProperties(fname=path)
                selected_name = prop.get_name()
                selected_path = path
                break
            except Exception:
                pass

if selected_name is not None:

```

```

# 찾은 폰트를 명시적으로 등록
if selected_path:
    try:
        font_manager.fontManager.addfont(selected_path)
    except Exception:
        pass

rcParams["font.family"] = selected_name
rcParams["axes.unicode_minus"] = False

if verbose:
    print(f"한글 폰트 설정 완료: {selected_name}")
    print(f"폰트 경로: {selected_path}")
else:
    rcParams["axes.unicode_minus"] = False
    if verbose:
        print("△ 한글 폰트를 자동 탐색하지 못함.")
        print("Windows: Malgun Gothic / Linux: NanumGothic 또는 Noto Sans CJK 설치 권장")

return selected_name, selected_path

KOREAN_FONT_NAME, KOREAN_FONT_PATH = setup_korean_font()

# Seaborn은 Matplotlib 설정을 그대로 사용하도록 theme만 적용
sns.set_theme(
    style="whitegrid",
    rc={
        "font.family": KOREAN_FONT_NAME if KOREAN_FONT_NAME else rcParams["font.family"],
        "axes.unicode_minus": False
    }
)

print("Matplotlib font.family:", rcParams["font.family"])

```

한글 폰트 설정 완료: NanumGothic

폰트 경로: /usr/share/fonts/truetype/nanum/NanumGothicBold.ttf

Matplotlib font.family: ['NanumGothic']

문서 표기 규칙

이 노트북에서는 수식과 코드의 의미를 빠르게 구분할 수 있도록 다음 표기 사용함.

표기	의미
<code>x</code> , <code>W</code> , <code>b</code>	Python 코드 또는 변수명
x, W, b	수학적 기호
$\hat{y}$	모델의 예측값
y	실제 정답
z	활성화 함수에 들어가기 전 가중합 또는 logit
a	활성화 함수 적용 후 값
L	Loss
η	Learning Rate
$\frac{\partial L}{\partial w}$	특정 가중치에 대한 Loss의 Gradient

핵심 읽기 원칙

코드는 무엇을 실행하는가, 수식은 그 코드가 어떤 계산을 하는가를 보여주는 표현임.

예를 들어 다음 두 표현은 같은 계산을 의미함.

```
z = W @ x + b
```

$$z = Wx + b$$

@는 Python/NumPy/PyTorch에서 행렬곱을 의미함.

Part 1 . 최종 문제부터 보기 — 왜 Perceptron으로 부족한가?

제공 자료의 Perceptron 과제는 `make_moons` 를 사용함. 두 클래스가 초승달처럼 휘어 있기 때문에 선형 결정 경계 하나만으로 완벽하게 분리하기 어려운 데이터임.

이 데이터가 좋은 이유:

- Feature가 2 개라서 좌표평면에 직접 표시 가능
- 클래스가 2 개라서 분류 구조가 단순함
- 비선형 경계가 필요하므로 Perceptron → MLP의 필요성을 눈으로 확인 가능

먼저 데이터를 보고, “어떤 모양의 경계가 필요한가?”부터 판단함.

```
# 1. make_moons 생성
X, y = make_moons(n_samples=600, noise=0.20, random_state=SEED)
moons_df = pd.DataFrame(X, columns=["feature_1", "feature_2"])
moons_df["class"] = y

print(moons_df.head())
print("\n클래스 개수")
print(moons_df["class"].value_counts().sort_index())

plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=moons_df, x="feature_1", y="feature_2",
    hue="class", s=55, alpha=0.8
)
plt.title("Top-Down 출발점: make_moons 이진 분류 데이터")
plt.show()
```

	feature_1	feature_2	class
0	0.5810	1.0143	0
1	0.6961	-0.4370	1
2	1.6490	0.0736	1
3	0.8674	0.6769	0
4	-0.3161	0.7105	0

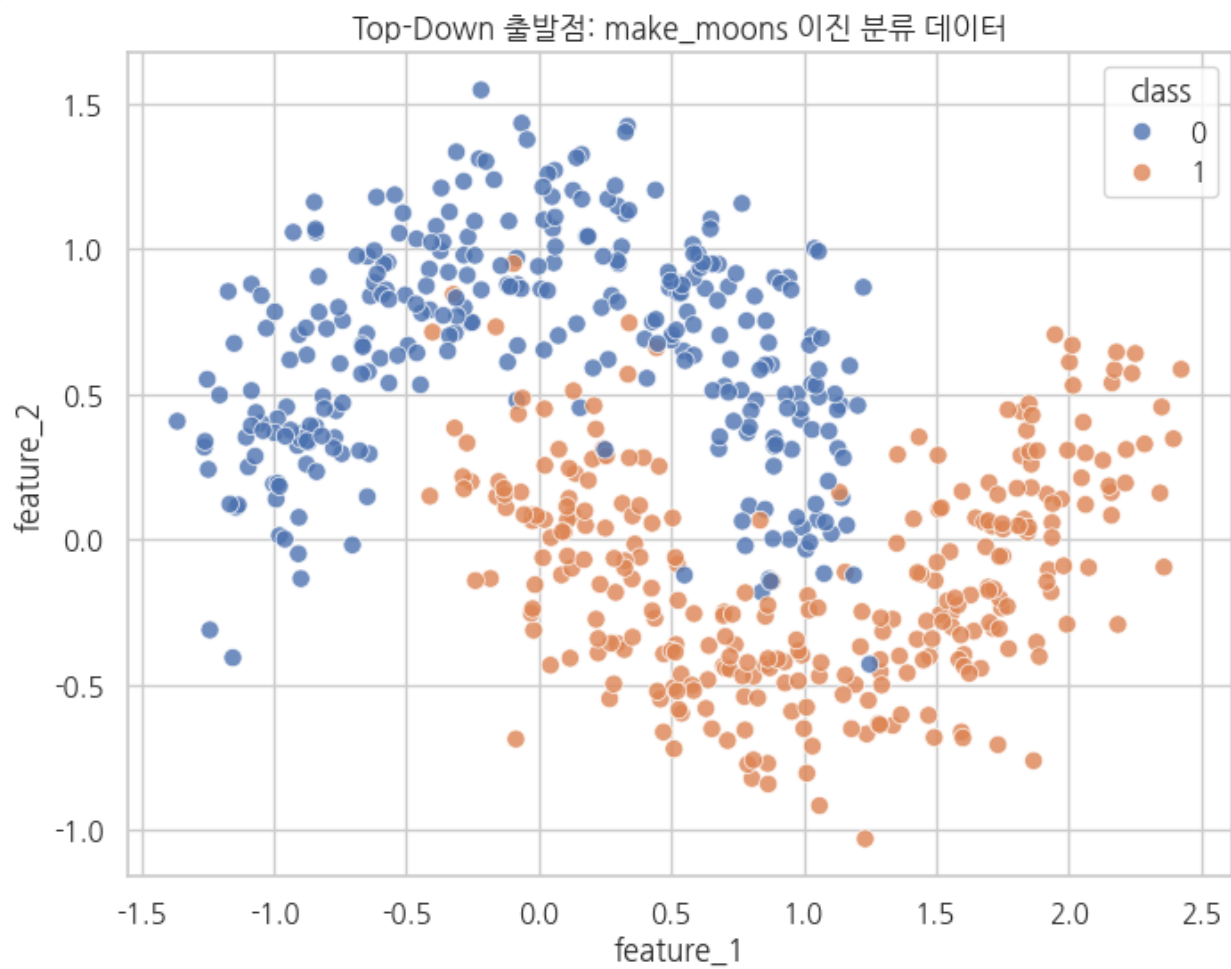
클래스 개수

class

0 300

1 300

Name: count, dtype: int64



관찰해야 할 것

점의 색을 구분하는 직선 하나를 상상해보면 완벽하게 나눌 수 없음.

Perceptron의 핵심 계산은 다음과 같음.

$$z = w_1x_1 + w_2x_2 + b$$

그리고 $z = 0$ 인 위치가 결정 경계임.

$$w_1x_1 + w_2x_2 + b = 0$$

Feature가 2 개라면 이 식은 직선임. 즉 단일 Perceptron은 직선 하나로 공간을 나눔.

핵심 결론

MLP가 필요한 이유

Linear 층만 여러 개 연결하면 전체 계산을 다시 하나의 Linear 변환으로 합칠 수 있음.

따라서 중간에 ReLU 같은 비선형 활성화 함수가 반드시 필요함.

$$\text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear}$$

이 구조부터 단일 직선으로 표현할 수 없는 패턴을 학습할 수 있게 됨.

Part 2 . MLP 전체 구조를 먼저 본 뒤 한 층씩 분해하기

이번 이진 분류 모델:

입력 $x=(x_1, x_2)$



Linear($2 \rightarrow 8$)



ReLU



Linear($8 \rightarrow 8$)



ReLU



Linear($8 \rightarrow 1$)



logit z



Sigmoid



$P(\text{class}=1)$

PyTorch에서는 학습 안정성을 위해 보통 마지막 **Sigmoid** 를 모델 안에 직접 넣지 않고

BCEWithLogitsLoss 를 사용함.

즉:

모델 출력 = logit

BCEWithLogitsLoss(logit, y)

↳ 내부적으로 sigmoid + binary cross entropy 수행

이 방식이 큰 양수/음수 logit에서도 수치적으로 더 안정적임. 추가 해설임.

2. 구조를 코드로 먼저 확인

```
class MoonMLP(nn.Module):  
    def __init__(self):  
        super().__init__()
```

```

        self.net = nn.Sequential(
            nn.Linear(2, 8),
            nn.ReLU(),
            nn.Linear(8, 8),
            nn.ReLU(),
            nn.Linear(8, 1)
        )

    def forward(self, x):
        return self.net(x).squeeze(1)

model_preview = MoonMLP()
print(model_preview)

dummy = torch.tensor([[0.2, -0.5], [1.0, 0.3]], dtype=torch.float32)
print("\n입력 shape :", dummy.shape)
print("출력 shape :", model_preview(dummy).shape)

```

```

MoonMLP(
  (net): Sequential(
    (0): Linear(in_features=2, out_features=8, bias=True)
    (1): ReLU()
    (2): Linear(in_features=8, out_features=8, bias=True)
    (3): ReLU()
    (4): Linear(in_features=8, out_features=1, bias=True)
  )
)

```

```

입력 shape : torch.Size([2, 2])
출력 shape : torch.Size([2])

```

Shape를 읽는 법

`Linear(2, 8)`은 “데이터 1 개가 가진 Feature 2 개를 새로운 값 8 개로 변환”함.

배치가 3 2 개라면:

$$(32, 2) \rightarrow (32, 8)$$

다음 `Linear(8,8)`:

$$(32, 8) \rightarrow (32, 8)$$

마지막 `Linear(8,1)`:

$$(32, 8) \rightarrow (32, 1)$$

여기서 8은 “정답 클래스 8 개”가 아님. 은닉 표현(hidden representation)의 차원 수임.

은닉 뉴런들은 원래 Feature를 조합하여 새로운 특징을 만드는 계산 노드로 이해하면 됨.

Part 기

3 . 가장 작은 MLP 하나를 손으로 계산하

학습 전체를 이해하기 전에 **샘플 1** 개의 계산을 숫자로 끝까지 따라감.

구조를 의도적으로 작게 설정함.

입력 2개 → 은닉 뉴런 3개 → 출력 1개

입력:

$$x = \begin{bmatrix} 0.5 \\ -1.0 \end{bmatrix}$$

첫 번째 층:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

활성화:

$$a^{(1)} = \text{ReLU}(z^{(1)})$$

출력층:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

확률:

$$p = \sigma(z^{(2)}) = \frac{1}{1 + e^{-z^{(2)}}}$$

기호 정리

기호	읽는 법	의미	코드 대응
x	엑스	입력 Feature 벡터	<code>x</code>
$W^{(1)}$	더블유 1 층	첫 번째 층 Weight 행렬	<code>W1</code>

기호	읽는 법	의미	코드 대응
$b^{(1)}$	비 1 층	첫 번째 층 Bias	b1
$z^{(1)}$	지 1 층	첫 층 가중합	z1
$a^{(1)}$	에이 1 층	ReLU 통과 후 활성화값	a1
$z^{(2)}$	지 2 층	최종 출력 Logit	z2
p	피	Class 1 확률	p
L	엘	Loss	loss

한 샘플의 전체 계산식

$$x \xrightarrow{W^{(1)}x+b^{(1)}} z^{(1)} \xrightarrow{\text{ReLU}} a^{(1)} \xrightarrow{W^{(2)}a^{(1)}+b^{(2)}} z^{(2)} \xrightarrow{\sigma} p \xrightarrow{\text{BCE}} L$$

```
# 3. 직접 계산할 작은 MLP의 고정 값
x = np.array([0.5, -1.0])
y_true = 1.0

W1 = np.array([
    [ 0.8, -0.4],
    [ 0.3,  0.9],
    [-0.5,  0.2]
])
b1 = np.array([0.1, -0.2, 0.05])

W2 = np.array([1.2, -0.7, 0.5])
b2 = -0.1

z1 = W1 @ x + b1
a1 = np.maximum(0, z1)
z2 = W2 @ a1 + b2
p = 1 / (1 + np.exp(-z2))
loss = -(y_true*np.log(p) + (1-y_true)*np.log(1-p))

calc = pd.DataFrame({
    "뉴런": ["h1", "h2", "h3"],
    "가중합 z1": z1,
    "ReLU 후 a1": a1
})
display(calc)

print(f"출력 logit z2 = {z2:.6f}")
```

```
print(f"sigmoid 확률 p = {p:.6f}")
print(f"BCE loss      = {loss:.6f}")
```

	뉴런	가중합 z 1	ReLU 후 a 1
0	h 1	0 . 9 0	0 . 9
1	h 2	- 0 . 9 5	0 . 0
2	h 3	- 0 . 4 0	0 . 0

출력 logit z2 = 0.980000
sigmoid 확률 p = 0.727108
BCE loss = 0.318680

각 숫자의 의미

1) $W^{(1)}x$

한 은닉 뉴런이 입력 Feature 2 개를 **가중합**함.

예를 들어 첫 번째 뉴런:

$$z_1^{(1)} = 0.8(0.5) + (-0.4)(-1.0) + 0.1$$

- **0.8**: 첫 번째 Feature의 영향
- **-0.4**: 두 번째 Feature의 영향
- **0.1**: bias
- 결과 z : 활성화 함수에 들어가기 전의 원시 점수

2) ReLU

$$\text{ReLU}(z) = \max(0, z)$$

음수는 0 으로 버리고 양수는 유지함.

여기서 중요한 점은 단순한 “음수 제거”가 아니라 **비선형 변환**이라는 사실임.

중간에 ReLU가 없으면:

$$W_2(W_1x + b_1) + b_2$$

를 전개하여 다시:

$$W'x + b'$$

형태로 합칠 수 있음. 결국 직선 경계로 되돌아감.

3) logit

마지막 Linear 출력 $z^{(2)}$ 는 아직 확률이 아님.

- 큰 양수: class 1 쪽 강한 증거

- 큰 음수: class 0 쪽 강한 증거
- 0 근처: 애매함

4) sigmoid

logit을 0 ~ 1 사이 값으로 바꿈.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

확률처럼 해석 가능한 값을 얻음.

5) BCE

정답이 1 일 때:

$$L = -\log(p)$$

모델이 정답 1 에 높은 확률을 주면 loss가 작아짐.

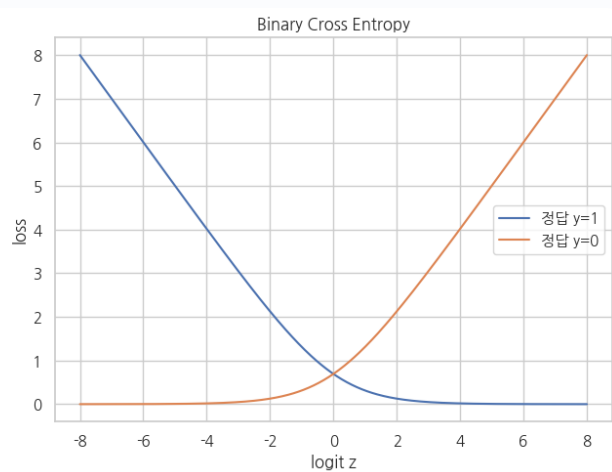
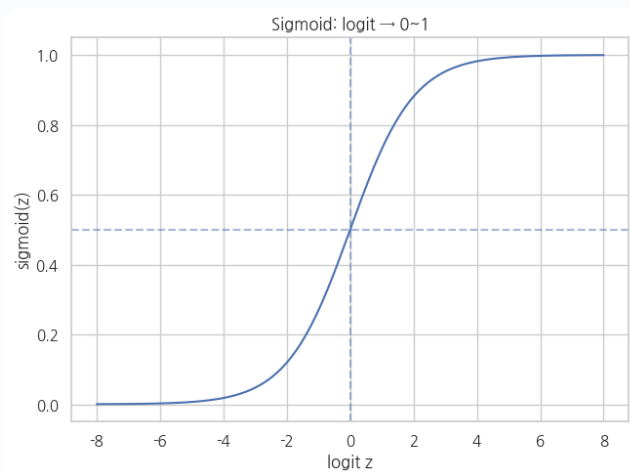
```
# 4. sigmoid와 BCE의 의미를 시각화
z_grid = np.linspace(-8, 8, 400)
sigmoid = 1 / (1 + np.exp(-z_grid))
loss_y1 = -np.log(np.clip(sigmoid, 1e-12, 1))
loss_y0 = -np.log(np.clip(1 - sigmoid, 1e-12, 1))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.lineplot(x=z_grid, y=sigmoid, ax=axes[0])
axes[0].axhline(0.5, linestyle="--", alpha=0.5)
axes[0].axvline(0, linestyle="--", alpha=0.5)
axes[0].set_title("Sigmoid: logit → 0~1")
axes[0].set_xlabel("logit z")
axes[0].set_ylabel("sigmoid(z)")

sns.lineplot(x=z_grid, y=loss_y1, label="정답 y=1", ax=axes[1])
sns.lineplot(x=z_grid, y=loss_y0, label="정답 y=0", ax=axes[1])
axes[1].set_title("Binary Cross Entropy")
axes[1].set_xlabel("logit z")
axes[1].set_ylabel("loss")

plt.tight_layout()
plt.show()
```



Part 4 . Backpropagation — “틀렸으니 어느 가중치를 얼마나 바꿀까?”

제공 자료에서는 `autograd` 가 순전파 연산을 추적하고 `.backward()` 호출 시 chain rule을 이용해 기울기를 계산한다고 설명함.

여기서는 그 기울기가 실제로 무엇인지 작은 모델로 확인함.

4 - 1 . 출력층에서 시작

Sigmoid + BCE 조합에서는 출력 logit에 대한 미분이 매우 단순해짐.

$$\frac{\partial L}{\partial z^{(2)}} = p - y$$

이를 $\delta^{(2)}$ 라고 둬.

$$\delta^{(2)} = p - y$$

출력층 가중치:

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} a^{(1)}$$

출력층 bias:

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

4 - 2 . 은닉층으로 전달

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \text{ReLU}'(z^{(1)})$$

여기서 $\odot$ 는 원소별 곱임.

ReLU 미분:

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

즉 ReLU에서 0 이 된 뉴런은 이번 샘플의 역전파에서도 gradient가 0 이 될 수 있음.

첫 층:

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T$$

이 식 때문에 각 입력값 $\mathcal{X}$ 가 가중치 업데이트 크기에 직접 영향을 줌.

Chain Rule을 문장으로 읽기

역전파는 다음 질문을 뒤에서부터 반복하는 과정임.

1. 출력값이 조금 변하면 Loss가 얼마나 변하는가?
2. 은닉값이 조금 변하면 출력값이 얼마나 변하는가?
3. Weight가 조금 변하면 은닉값이 얼마나 변하는가?

이를 미분의 곱으로 연결함.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w}$$

중요

Backpropagation은 Weight를 직접 수정하는 과정이 아님.

Gradient를 계산하는 과정임.

실제 수정은 `optimizer.step()` 에서 수행함.

```
# 5. 위의 숫자로 gradient를 손 계산
delta2 = p - y_true
dW2 = delta2 * a1
db2 = delta2
```

```

relu_grad = (z1 > 0).astype(float)
delta1 = (W2 * delta2) * relu_grad
dW1 = np.outer(delta1, x)
db1 = delta1

print("delta2 =", delta2)
print("\ndW2 =", dW2)
print("db2 =", db2)
print("\nReLU' =", relu_grad)
print("delta1 =", delta1)
print("\ndW1 =")
print(dW1)
print("\ndb1 =", db1)

```

```

delta2 = -0.27289178365887046

dW2 = [-0.24560261 -0.          -0.          ]
db2  = -0.27289178365887046

ReLU' = [1.  0.  0.]
delta1 = [-0.32747014  0.          -0.          ]

dW1 =
[[-0.16373507  0.32747014]
 [ 0.          -0.          ]
 [-0.          0.          ]]

db1 = [-0.32747014  0.          -0.          ]

```

6. PyTorch autograd 결과와 직접 계산 비교

```

xt = torch.tensor(x, dtype=torch.float64)
yt = torch.tensor(y_true, dtype=torch.float64)

W1t = torch.tensor(W1, dtype=torch.float64, requires_grad=True)
b1t = torch.tensor(b1, dtype=torch.float64, requires_grad=True)
W2t = torch.tensor(W2, dtype=torch.float64, requires_grad=True)
b2t = torch.tensor(b2, dtype=torch.float64, requires_grad=True)

z1t = W1t @ xt + b1t
a1t = torch.relu(z1t)
z2t = W2t @ a1t + b2t

loss_t = torch.nn.functional.binary_cross_entropy_with_logits(z2t, yt)
loss_t.backward()

print("loss:", loss_t.item())
print("\n수동 dW2 :", dW2)

```

```
print("autograd dW2:", W2t.grad.detach().numpy())
print("\n수동 dW1:\n", dW1)
print("autograd dW1:\n", W1t.grad.detach().numpy())
```

loss: 0.3186799592371327

수동 dW2 : [-0.24560261 -0. -0.]
autograd dW2: [-0.24560261 -0. -0.]

수동 dW1:
[[-0.16373507 0.32747014]
[0. -0.]
[-0. 0.]]
autograd dW1:
[[-0.16373507 0.32747014]
[0. -0.]
[0. -0.]]

gradient의 부호를 읽는 법

Gradient는 “그 파라미터를 증가시키면 loss가 어느 방향으로 변하는가”를 나타냄.

Gradient Descent:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

- gradient가 양수 → w 를 감소시키는 방향
- gradient가 음수 → w 를 증가시키는 방향
- η → learning rate, 한 번에 움직이는 크기

중요:

gradient 자체가 “좋은 방향”이 아니라 loss가 증가하는 방향임. 그래서 optimizer는 gradient를 빼는 방향으로 이동함.

7. 단일 gradient step이 loss를 실제로 줄이는지 확인

lr = 0.1

W1_new = W1 - lr * dW1

b1_new = b1 - lr * db1

W2_new = W2 - lr * dW2

b2_new = b2 - lr * db2

z1_new = W1_new @ x + b1_new

a1_new = np.maximum(0, z1_new)

z2_new = W2_new @ a1_new + b2_new

p_new = 1/(1+np.exp(-z2_new))

loss_new = -(y_true*np.log(p_new)+(1-y_true)*np.log(1-p_new))

print(f"업데이트 전: p={p:.6f}, loss={loss:.6f}")

print(f"업데이트 후: p={p_new:.6f}, loss={loss_new:.6f}")

업데이트 전: p=0.727108, loss=0.318680

업데이트 후: p=0.753918, loss=0.282471

Part 5 . 실제 데이터 학습 준비 — 분할, 표준화, DataLoader

제공 자료의 공통 파이프라인:

```
Raw Data
  ↓
train / validation / test split
  ↓
StandardScaler
  ↓
TensorDataset
  ↓
DataLoader
  ↓
Model
```

왜 train에서만 **fit()** 하는가?

표준화:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

검증/테스트 데이터의 평균·표준편차까지 미리 사용하면 미래 정보를 학습 과정에 흘리는 data leakage가 발생함.

따라서:

```
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_valid = scaler.transform(X_valid)
X_test = scaler.transform(X_test)
```

이 순서가 중요함.

표준화 수식 읽기

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

기호	의미
x	원래 값
μ	학습 데이터 평균
σ	학습 데이터 표준편차
$x - \mu$	평균에서 얼마나 떨어져 있는지
$\frac{x - \mu}{\sigma}$	그 거리가 표준편차 몇 배인지

따라서 표준화 후에는 일반적으로:

$$\mu \approx 0, \quad \sigma \approx 1$$

이 됨.

```
# 8. train/valid/test = 70/15/15 분할
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=SEED
)
X_valid, X_test, y_valid, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_valid_s = scaler.transform(X_valid)
X_test_s = scaler.transform(X_test)

before_after = pd.DataFrame({
    "구분": ["원본 feature_1", "표준화 feature_1", "원본 feature_2", "표준화 feature_2"],
    "평균": [
        X_train[:,0].mean(), X_train_s[:,0].mean(),
        X_train[:,1].mean(), X_train_s[:,1].mean()
    ],
    "표준편차": [
        X_train[:,0].std(), X_train_s[:,0].std(),
        X_train[:,1].std(), X_train_s[:,1].std()
    ]
})
```

```

}))
display(before_after)

print("Train:", X_train_s.shape)
print("Valid:", X_valid_s.shape)
print("Test :", X_test_s.shape)

```

	구분	평균	표준편차
0	원본 feature_ 1	4 . 8 6 6 6 e- 0 1	0 . 8 9 6 9
1	표준화 feature_ 1	- 5 . 2 5 5 1 e- 1 6	1 . 0 0 0 0
2	원본 feature_ 2	2 . 8 6 4 0 e- 0 1	0 . 5 3 3 5
3	표준화 feature_ 2	9 . 5 1 6 2 e- 1 7	1 . 0 0 0 0

Train: (420, 2)
 Valid: (90, 2)
 Test : (90, 2)

9. 표준화 전/후 분포 비교

```

viz_before = pd.DataFrame({
    "value": np.r_[X_train[:,0], X_train[:,1]],
    "feature": ["feature_1"]*len(X_train) + ["feature_2"]*len(X_train),
    "stage": "before"
})

viz_after = pd.DataFrame({
    "value": np.r_[X_train_s[:,0], X_train_s[:,1]],
    "feature": ["feature_1"]*len(X_train_s) + ["feature_2"]*len(X_train_s),
    "stage": "after"
})

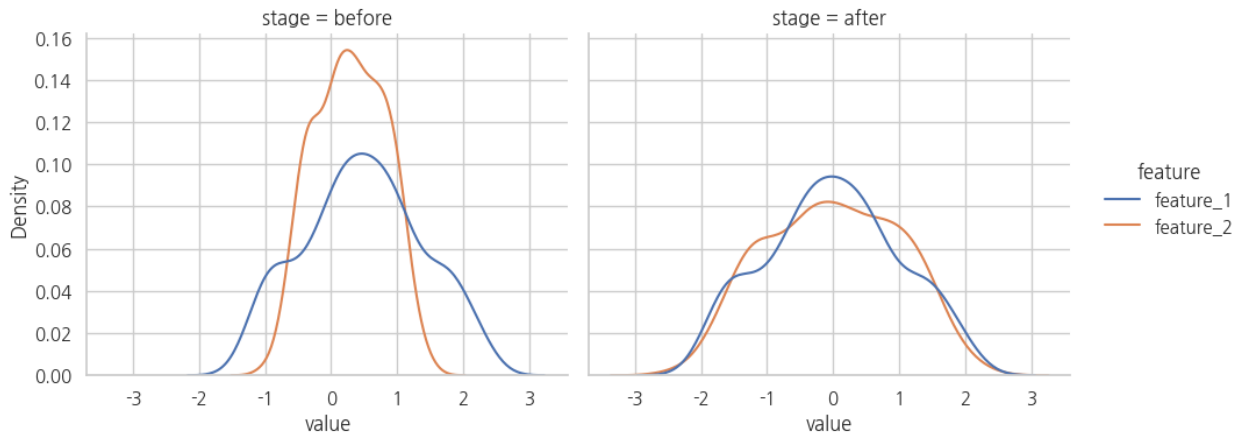
viz_scale = pd.concat([viz_before, viz_after], ignore_index=True)

g = sns.displot(
    data=viz_scale, x="value", hue="feature", col="stage",
    kind="kde", fill=False, height=4, aspect=1.2
)

g.fig.suptitle("표준화 전/후 Feature 분포", y=1.05)
plt.show()

```

표준화 전/후 Feature 분포



```
# 10. TensorDataset / DataLoader
```

```
def to_loader(X_arr, y_arr, batch_size=32, shuffle=False):
    X_t = torch.tensor(X_arr, dtype=torch.float32)
    y_t = torch.tensor(y_arr, dtype=torch.float32)
    ds = TensorDataset(X_t, y_t)
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle)
```

```
train_loader = to_loader(X_train_s, y_train, batch_size=32, shuffle=True)
valid_loader = to_loader(X_valid_s, y_valid, batch_size=64, shuffle=False)
test_loader = to_loader(X_test_s, y_test, batch_size=64, shuffle=False)
```

```
xb, yb = next(iter(train_loader))
print("한 배치 X shape:", xb.shape)
print("한 배치 y shape:", yb.shape)
print("첫 3개 X:\n", xb[:3])
print("첫 3개 y:", yb[:3])
```

```
한 배치 X shape: torch.Size([32, 2])
```

```
한 배치 y shape: torch.Size([32])
```

```
첫 3개 X:
```

```
tensor([[ -1.8866,  0.3946],
        [ 1.5149, -0.4900],
        [-0.1815, -0.7216]])
```

```
첫 3개 y: tensor([0., 1., 1.])
```

배치가 의미하는 것

전체 420 개 학습 샘플을 한 번에 처리할 수도 있지만, DataLoader가 32 개씩 나누면:

420개 데이터

↓

32개

32개

32개

...

마지막 작은 배치

각 배치마다:

forward

→ loss

→ backward

→ optimizer.step()

가 한 번 실행됨.

Epoch는 전체 학습 데이터를 한 바퀴 사용한 횟수임.

`batch_size=32` 이고 학습 데이터가 420 개라면 대략 한 epoch에 14 번의 파라미터 업데이트가 발생함.

Part 6 . Perceptron baseline과 MLP를 같은 조건에서 비교

Top-Down 질문:

비선형 데이터에서 MLP가 실제로 무엇을 더 잘하는가?

비교 모델:

Perceptron에 가까운 선형 모델

2 input $\rightarrow$ Linear(2,1)

MLP

2 $\rightarrow$ 8 $\rightarrow$ ReLU $\rightarrow$ 8 $\rightarrow$ ReLU $\rightarrow$ 1

같은 데이터, 같은 loss, 같은 optimizer 계열로 비교함.

```
# 11. 두 모델 정의
class LinearBinary(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(2, 1)
    def forward(self, x):
        return self.fc(x).squeeze(1)

class MLPBinary(nn.Module):
    def __init__(self, hidden=8):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(1)
```

```

def train_binary(model, train_loader, valid_loader, epochs=120, lr=0.01):
    model = model.to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCEWithLogitsLoss()
    hist = []

    for epoch in range(1, epochs+1):
        model.train()
        tr_loss = 0.0
        tr_correct = 0
        tr_total = 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
            opt.step()

            tr_loss += loss.item() * len(xb)
            pred = (logits >= 0).long()
            tr_correct += (pred == yb.long()).sum().item()
            tr_total += len(xb)

        model.eval()
        va_loss = 0.0
        va_correct = 0
        va_total = 0
        with torch.no_grad():
            for xb, yb in valid_loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                logits = model(xb)
                loss = criterion(logits, yb)
                va_loss += loss.item() * len(xb)
                pred = (logits >= 0).long()
                va_correct += (pred == yb.long()).sum().item()
                va_total += len(xb)

        hist.append({
            "epoch": epoch,
            "train_loss": tr_loss/tr_total,
            "train_acc": tr_correct/tr_total,
            "valid_loss": va_loss/va_total,
            "valid_acc": va_correct/va_total,
        })

    return model, pd.DataFrame(hist)

linear_model, linear_hist = train_binary(LinearBinary(), train_loader, valid_loader)
mlp_model, mlp_hist = train_binary(MLPBinary(hidden=8), train_loader, valid_loader)

```

```
print("Linear final valid acc:", linear_hist.iloc[-1]["valid_acc"])
print("MLP    final valid acc:", mlp_hist.iloc[-1]["valid_acc"])
```

```
Linear final valid acc: 0.8777777777777778
MLP    final valid acc: 0.9777777777777777
```

12. 학습 곡선 비교

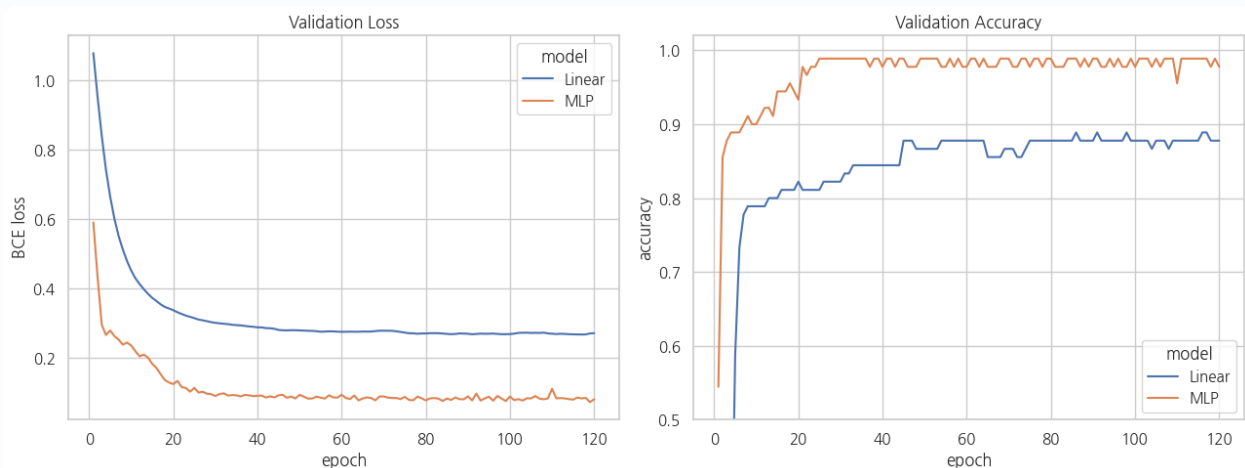
```
plot_hist = pd.concat([
    linear_hist.assign(model="Linear"),
    mlp_hist.assign(model="MLP")
], ignore_index=True)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.lineplot(data=plot_hist, x="epoch", y="valid_loss", hue="model", ax=axes[0])
axes[0].set_title("Validation Loss")
axes[0].set_ylabel("BCE loss")

sns.lineplot(data=plot_hist, x="epoch", y="valid_acc", hue="model", ax=axes[1])
axes[1].set_title("Validation Accuracy")
axes[1].set_ylabel("accuracy")
axes[1].set_ylim(0.5, 1.02)

plt.tight_layout()
plt.show()
```



그래프 해석

- Loss: 모델이 정답에 얼마나 잘 맞는 확률을 내놓는지 반영
- Accuracy: 최종 class 판단이 맞았는지의 비율
- Accuracy는 같아도 loss는 다를 수 있음

예:

정답 = 1

모델 A 확률 = 0.51 → 맞음

모델 B 확률 = 0.99 → 맞음

둘 다 accuracy에는 1 회 정답으로 동일하지만 BCE loss는 B가 훨씬 작음.

따라서 학습에서는 단순 정답 개수보다 **연속적으로 미분 가능한** loss가 필요함.

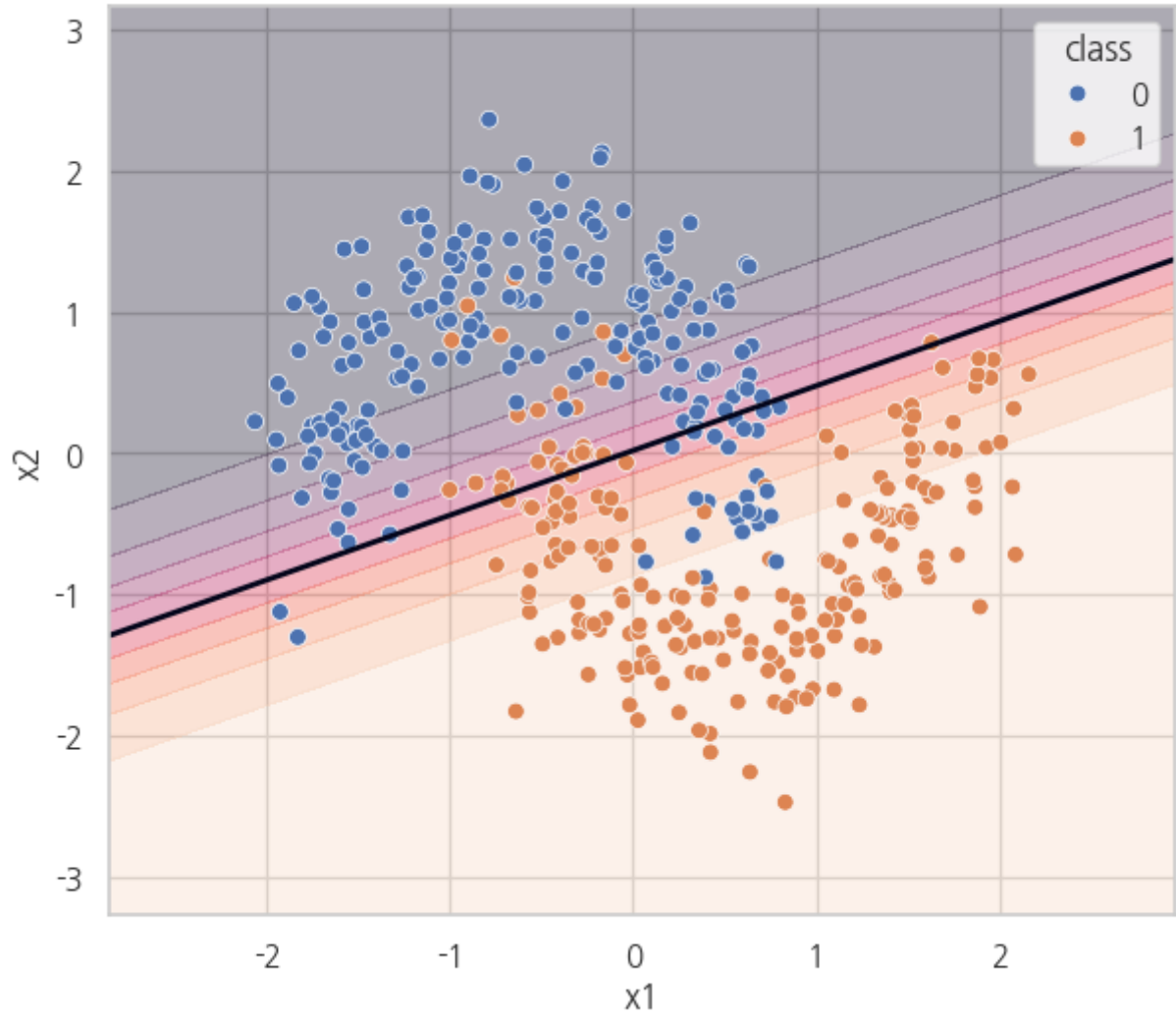
13. 결정 경계 시각화

```
def decision_surface(model, X_scaled, y_data, title):
    x1_min, x1_max = X_scaled[:,0].min()-0.8, X_scaled[:,0].max()+0.8
    x2_min, x2_max = X_scaled[:,1].min()-0.8, X_scaled[:,1].max()+0.8
    xx, yy = np.meshgrid(
        np.linspace(x1_min, x1_max, 260),
        np.linspace(x2_min, x2_max, 260)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]
    with torch.no_grad():
        logits = model(torch.tensor(grid, dtype=torch.float32, device=DEVICE))
        probs = torch.sigmoid(logits).cpu().numpy().reshape(xx.shape)

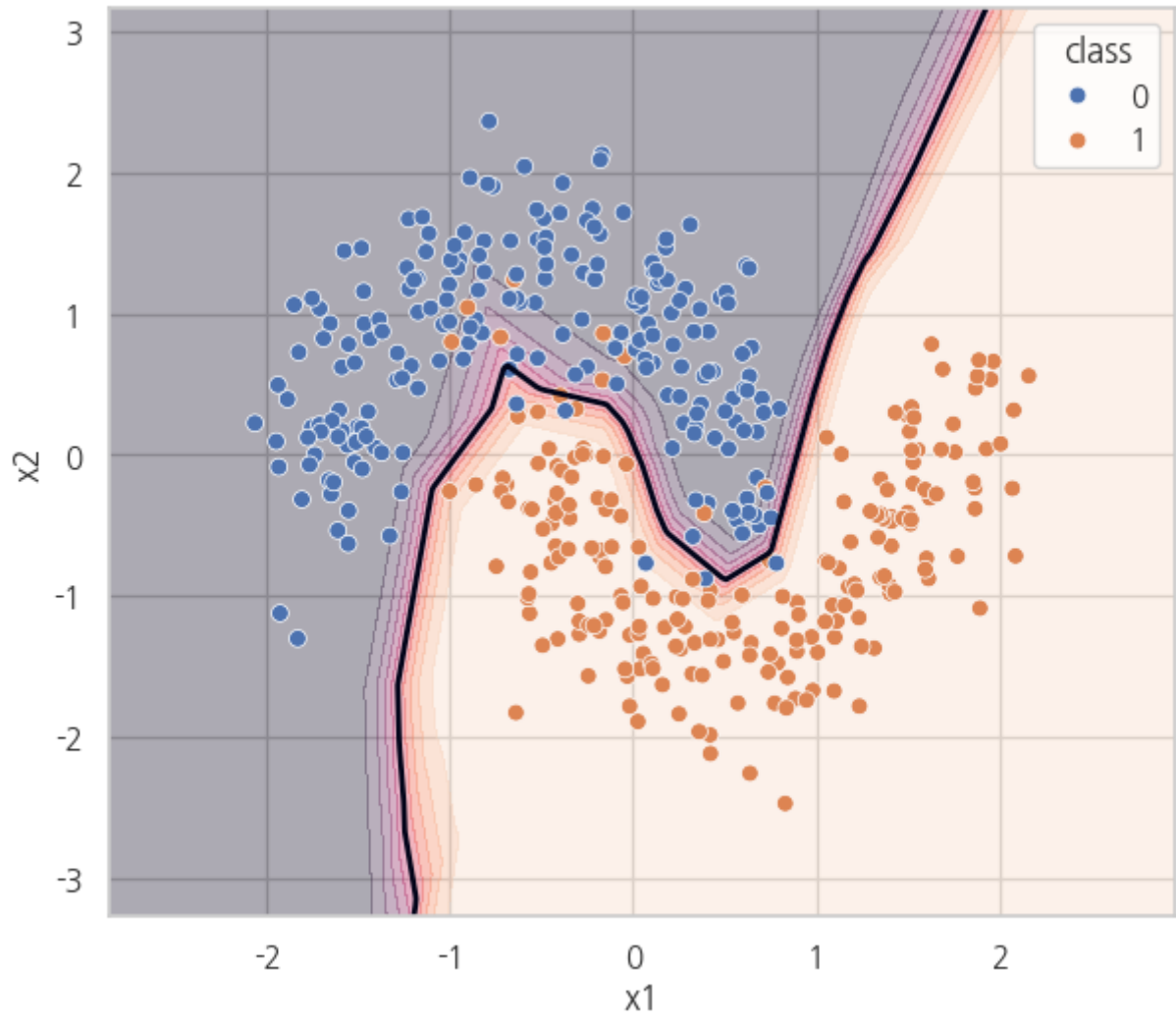
    plt.figure(figsize=(7, 6))
    plt.contourf(xx, yy, probs, levels=np.linspace(0,1,11), alpha=0.35)
    plt.contour(xx, yy, probs, levels=[0.5], linewidths=2)
    df = pd.DataFrame(X_scaled, columns=["x1", "x2"])
    df["class"] = y_data
    sns.scatterplot(data=df, x="x1", y="x2", hue="class", s=40)
    plt.title(title)
    plt.show()

decision_surface(linear_model, X_train_s, y_train, "Linear 모델: 직선 결정 경계")
decision_surface(mlp_model, X_train_s, y_train, "MLP: 비선형 결정 경계")
```


Linear 모델: 직선 결정 경계



MLP: 비선형 결정 경계



가장 중요한 시각적 결론

Linear 모델의 $z=0$ 경계는 직선임.

MLP에서는 여러 은닉 뉴런이 서로 다른 선형 변환을 만들고 ReLU가 영역별로 활성/비활성을 바꿈. 그 결과 최종 출력은 여러 piecewise-linear 영역의 조합이 되어 꺾인 경계를 표현 가능함.

“MLP가 선을 실제 곡선 함수로 직접 그린다”라고 이해하기보다는:

여러 선형 조각을 비선형 활성화로 조합해 복잡한 경계를 근사한다

가 더 정확함.

Part 7. 은닉 뉴런은 무엇을 보고 있는가?

MLP의 첫 번째 은닉층에는 8 개 뉴런이 있음.

각 뉴런:

$$z_j = w_{j1}x_1 + w_{j2}x_2 + b_j$$

$$a_j = \max(0, z_j)$$

즉 각 뉴런은 입력 평면에서 자기만의 선형 경계를 가지고 한쪽 영역에서 주로 활성화됨.

이를 실제 학습된 MLP에서 시각화함.

```
# 14. 첫 은닉층 activation map
mlp_model.eval()
first_linear = mlp_model.net[0]

x1_min, x1_max = X_train_s[:,0].min()-0.6, X_train_s[:,0].max()+0.6
x2_min, x2_max = X_train_s[:,1].min()-0.6, X_train_s[:,1].max()+0.6
xx, yy = np.meshgrid(np.linspace(x1_min,x1_max,180), np.linspace(x2_min,x2_max,180))
```

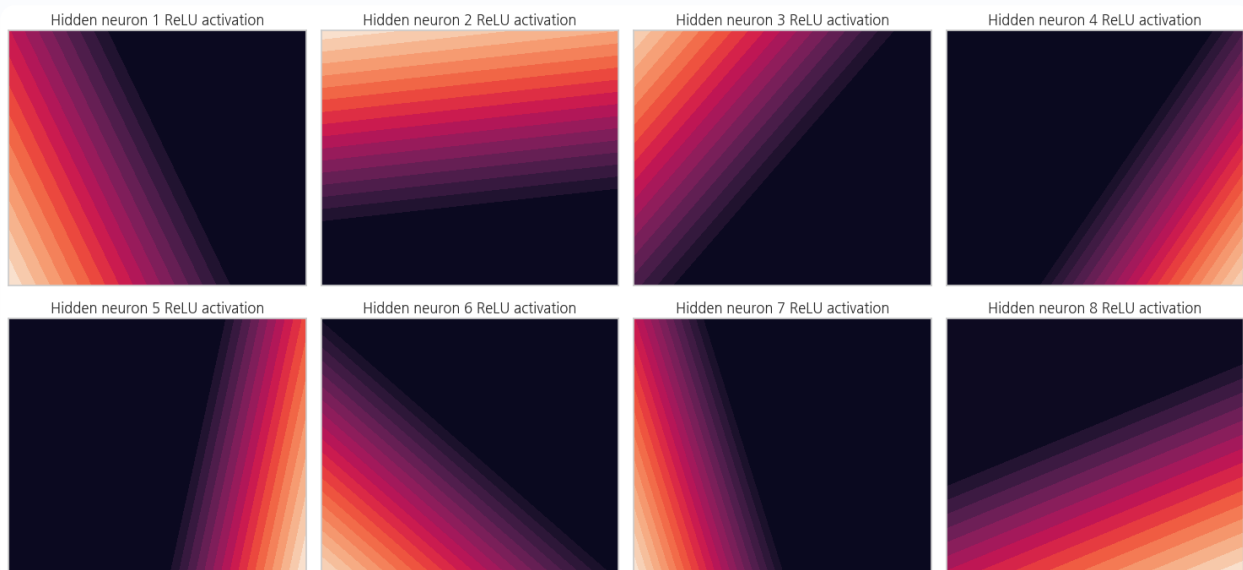
```

grid = np.c_[xx.ravel(), yy.ravel()]

with torch.no_grad():
    z = first_linear(torch.tensor(grid, dtype=torch.float32, device=DEVICE))
    a = torch.relu(z).cpu().numpy()

fig, axes = plt.subplots(2, 4, figsize=(15, 7))
for j, ax in enumerate(axes.flat):
    contour = ax.contourf(xx, yy, a[:,j].reshape(xx.shape), levels=20)
    ax.set_title(f"Hidden neuron {j+1} ReLU activation")
    ax.set_xticks([])
    ax.set_yticks([])
plt.tight_layout()
plt.show()

```



```

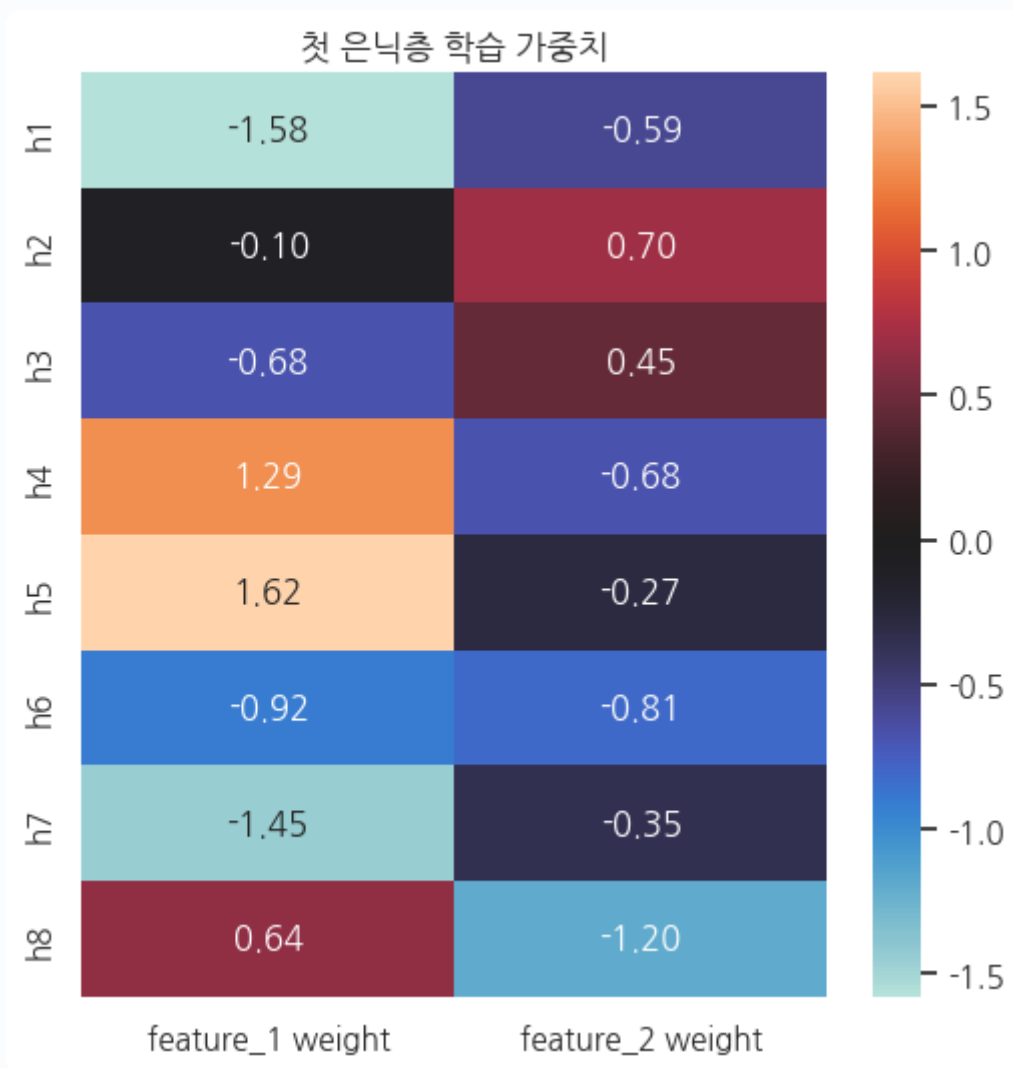
# 15. 첫 번째 층 weight heatmap
W = first_linear.weight.detach().cpu().numpy()
b = first_linear.bias.detach().cpu().numpy()

w_df = pd.DataFrame(W, columns=["feature_1 weight", "feature_2 weight"],
                    index=[f"h{i+1}" for i in range(W.shape[0])])
display(w_df.assign(bias=b))

plt.figure(figsize=(6, 6))
sns.heatmap(w_df, annot=True, fmt=".2f", center=0)
plt.title("첫 은닉층 학습 가중치")
plt.show()

```

	feature_1 weight	feature_2 weight	bias
h 1	- 1 . 5 7 8 7	- 0 . 5 8 6 0	0 . 7 7 5 1
h 2	- 0 . 0 9 9 4	0 . 6 9 7 3	1 . 0 0 9 9
h 3	- 0 . 6 7 6 2	0 . 4 5 1 7	0 . 2 6 6 5
h 4	1 . 2 9 0 5	- 0 . 6 7 6 3	- 0 . 6 0 5 7
h 5	1 . 6 1 8 4	- 0 . 2 7 2 5	- 1 . 0 5 5 7
h 6	- 0 . 9 1 6 1	- 0 . 8 0 8 0	0 . 1 5 5 8
h 7	- 1 . 4 4 8 3	- 0 . 3 5 0 9	- 0 . 7 8 4 8
h 8	0 . 6 4 2 4	- 1 . 2 0 2 6	0 . 8 9 0 0



Weight를 어떻게 읽을까?

예를 들어 특정 은닉 뉴런의 가중치가:

```
feature_1: +1.5  
feature_2: -0.4  
bias      : +0.2
```

라면 대략:

$$z = 1.5x_1 - 0.4x_2 + 0.2$$

를 계산함.

단, 은닉 뉴런 하나의 가중치를 곧바로 “feature_1이 전체 모델에서 1.5만큼 중요하다”라고 해석하면 안 됨.

이유:

- 뒤에 다른 은닉층이 존재
- ReLU로 활성화 여부가 샘플마다 달라짐
- 다음 층 가중치와 조합되어 최종 결과가 만들어짐

즉 선형회귀의 회귀계수처럼 직접적인 전역 해석이 어려움.

Part 8 . 학습 루프를 한 줄씩 의미로 읽기

제공 자료의 핵심 학습 순서:

```
model.train()

for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)

    logits = model(xb)          # 1. 순전파
    optimizer.zero_grad()       # 2. 이전 gradient 제거
    loss = criterion(logits, yb) # 3. 오차 측정
    loss.backward()             # 4. gradient 계산
    optimizer.step()            # 5. parameter 업데이트
```

왜 `zero_grad()` 가 필요한가?

PyTorch는 기본적으로 gradient를 누적(accumulate)함.

두 번 backward하면:

현재 grad = 이전 grad + 새 grad

일반적인 mini-batch 학습에서는 각 배치의 gradient로 한 번 업데이트하려 하므로 매 배치마다 지움.

왜 `model.train()` / `model.eval()` 이 필요한가?

Dropout/BatchNorm이 모드에 따라 다르게 동작하기 때문.

- `train()` → Dropout 활성화
- `eval()` → Dropout 비활성

제공 MLP 실습의 3-layer 모델은 Dropout을 사용하므로 특히 중요함.

Part 9 . Dropout을 눈으로 확인

Dropout은 학습 중 일부 activation을 확률적으로 0으로 만들어 특정 뉴런 조합에 지나치게 의존하는 현상을 줄임.

평가에서는 모든 뉴런을 사용함.

아래에서 같은 입력을 여러 번 넣어도 `train()`에서는 결과가 달라지고, `eval()`에서는 동일한 결과가 나오는 것을 확인함.

```
# 16. Dropout 동작 실험
drop_demo = nn.Sequential(
    nn.Linear(2, 6),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(6, 1)
)

sample = torch.tensor([[0.5, -0.2]], dtype=torch.float32)

drop_demo.train()
train_outputs = [drop_demo(sample).item() for _ in range(6)]

drop_demo.eval()
eval_outputs = [drop_demo(sample).item() for _ in range(6)]

print("train() 출력:", np.round(train_outputs, 4))
print("eval() 출력 :", np.round(eval_outputs, 4))
```

```
train() 출력: [-0.225 -0.1071 -0.0439 -0.1071 -0.0439 -0.2584]
eval() 출력 : [-0.2101 -0.2101 -0.2101 -0.2101 -0.2101 -0.2101]
```


Part 1 0. 실제 샘플 문제 — 8×8 손글씨 숫자 분류

제공 MLP 실습은 `sklearn.datasets.load_digits` 를 사용함.

- 샘플 수: 약 1,800 개
- 입력: 8×8 grayscale 이미지
- MLP에 넣기 위해 64 개 숫자로 펼침
- 출력 class: 0 ~ 9, 총 10 개

이제 앞에서 배운 구조를 그대로 확장함.

```
64 features
  ↓
Linear(64 → 128)
  ↓
ReLU
  ↓
Dropout
  ↓
Linear(128 → 64)
  ↓
ReLU
  ↓
Dropout
  ↓
Linear(64 → 10)
  ↓
10개 logits
  ↓
CrossEntropyLoss
```

```
# 17. digits 데이터 확인
digits = load_digits()
Xd = digits.data.astype(np.float32)
```

```

yd = digits.target.astype(np.int64)

print("X shape:", Xd.shape)
print("y shape:", yd.shape)
print("pixel min/max:", Xd.min(), Xd.max())

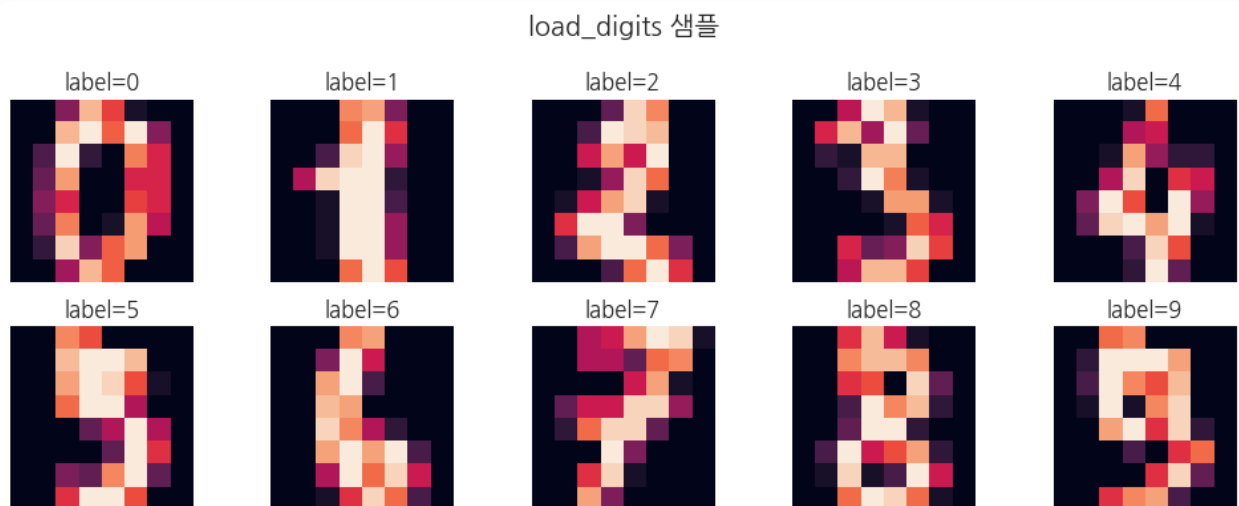
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for cls, ax in enumerate(axes.flat):
    idx = np.where(yd == cls)[0][0]
    ax.imshow(Xd[idx].reshape(8,8))
    ax.set_title(f"label={cls}")
    ax.axis("off")
plt.suptitle("load_digits 샘플")
plt.tight_layout()
plt.show()

```

```

X shape: (1797, 64)
y shape: (1797,)
pixel min/max: 0.0 16.0

```



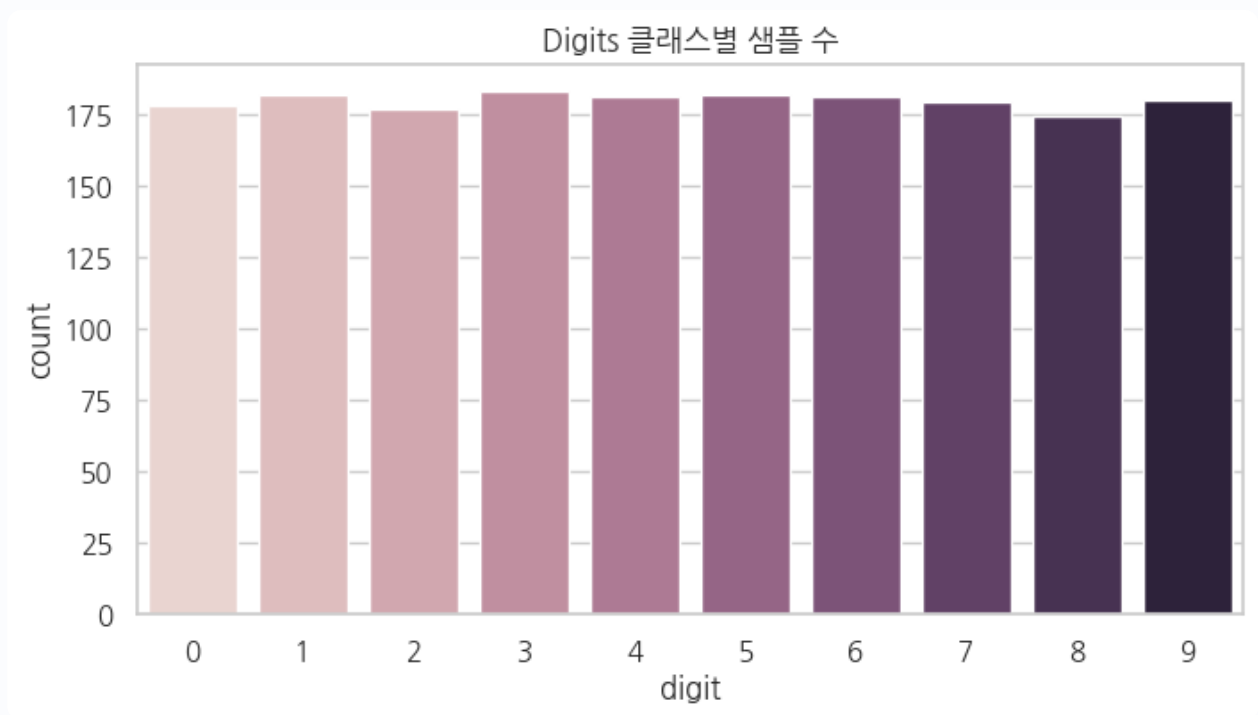
```

# 18. 클래스 분포
class_counts = pd.Series(yd).value_counts().sort_index().rename_axis("digit").reset_index(name="count")
display(class_counts)

plt.figure(figsize=(8,4))
sns.barplot(data=class_counts, x="digit", y="count", hue="digit", legend=False)
plt.title("Digits 클래스별 샘플 수")
plt.show()

```

	digit	count
0	0	1 7 8
1	1	1 8 2
2	2	1 7 7
3	3	1 8 3
4	4	1 8 1
5	5	1 8 2
6	6	1 8 1
7	7	1 7 9
8	8	1 7 4
9	9	1 8 0



6 4 개 Feature는 무엇인가?

8×8 이미지:

$$8 \times 8 = 64$$

각 픽셀 밝기가 하나의 Feature임.

2D 이미지

[8,8]

↓ flatten

[64]

↓ Linear

[128]

MLP는 이미지의 2 차원 공간 구조를 직접 이용하지 않고 일렬로 펼친 64 차원 벡터로 처리함.

이 때문에 이미지에서는 CNN이 더 적합한 경우가 많지만, 신경망 기본 원리 학습에는 작은 MLP가 적합함.

```
# 19. digits 분할/표준화
Xd_train, Xd_temp, yd_train, yd_temp = train_test_split(
    Xd, yd, test_size=0.20, random_state=SEED, stratify=yd
)
Xd_valid, Xd_test, yd_valid, yd_test = train_test_split(
    Xd_temp, yd_temp, test_size=0.50, random_state=SEED, stratify=yd_temp
)

digit_scaler = StandardScaler()
Xd_train_s = digit_scaler.fit_transform(Xd_train).astype(np.float32)
Xd_valid_s = digit_scaler.transform(Xd_valid).astype(np.float32)
Xd_test_s = digit_scaler.transform(Xd_test).astype(np.float32)

def multiclass_loader(X_arr, y_arr, batch_size=64, shuffle=False):
    ds = TensorDataset(
        torch.tensor(X_arr, dtype=torch.float32),
        torch.tensor(y_arr, dtype=torch.long)
    )
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle)

dtrain_loader = multiclass_loader(Xd_train_s, yd_train, shuffle=True)
```

```
dvalid_loader = multiclass_loader(Xd_valid_s, yd_valid)
dtest_loader = multiclass_loader(Xd_test_s, yd_test)

print(len(Xd_train_s), len(Xd_valid_s), len(Xd_test_s))
```

```
1437 180 180
```

Part 1 1. 다중 클래스에서는 왜 출력이 10 개인가?

숫자 0 ~ 9 각각에 대한 logit 10 개를 출력함.

예:

```
class 0 logit: -1.2
class 1 logit: 0.4
class 2 logit: 3.8 ← 가장 큼
...
class 9 logit: -0.7
```

예측:

```
pred = logits.argmax(dim=1)
```

즉 가장 큰 logit의 class index 선택.

Softmax

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

10 개 logit을 10 개 확률로 변환하고 전체 합을 1로 만듦.

`CrossEntropyLoss` 는 PyTorch에서 raw logits를 입력받아 내부적으로

`log_softmax + NLLLoss` 에 해당하는 계산을 안정적으로 처리함.

따라서 모델 끝에 `Softmax` 를 별도로 붙이지 않는 것이 일반적임.

Softmax 수식 해체

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

부분	의미
z_k	k 번째 클래스의 Logit
e^{z_k}	Logit을 양수 크기로 변환
$\sum_j e^{z_j}$	모든 클래스의 값을 합산
p_k	전체 중 k 번째 클래스가 차지하는 비율

결과적으로:

$$0 < p_k < 1$$

그리고:

$$\sum_{k=1}^K p_k = 1$$

이므로 확률 분포처럼 해석 가능함.

```
# 20. softmax 한 샘플
example_logits = torch.tensor([[0.3, -0.5, 2.1, 0.7]])
example_prob = torch.softmax(example_logits, dim=1)

prob_df = pd.DataFrame({
    "class": [0,1,2,3],
    "logit": example_logits.numpy().ravel(),
    "softmax_probability": example_prob.numpy().ravel()
})
display(prob_df)
print("확률 합:", example_prob.sum().item())
print("예측 class:", example_logits.argmax(dim=1).item())
```

	class	logit	softmax_probability
0	0	0.3	0.1112
1	1	-0.5	0.0500
2	2	2.1	0.6729
3	3	0.7	0.1659

확률 합: 0.999998807907104

예측 class: 2

Cross Entropy의 의미

정답 class가 t 라면:

$$L = -\log(p_t)$$

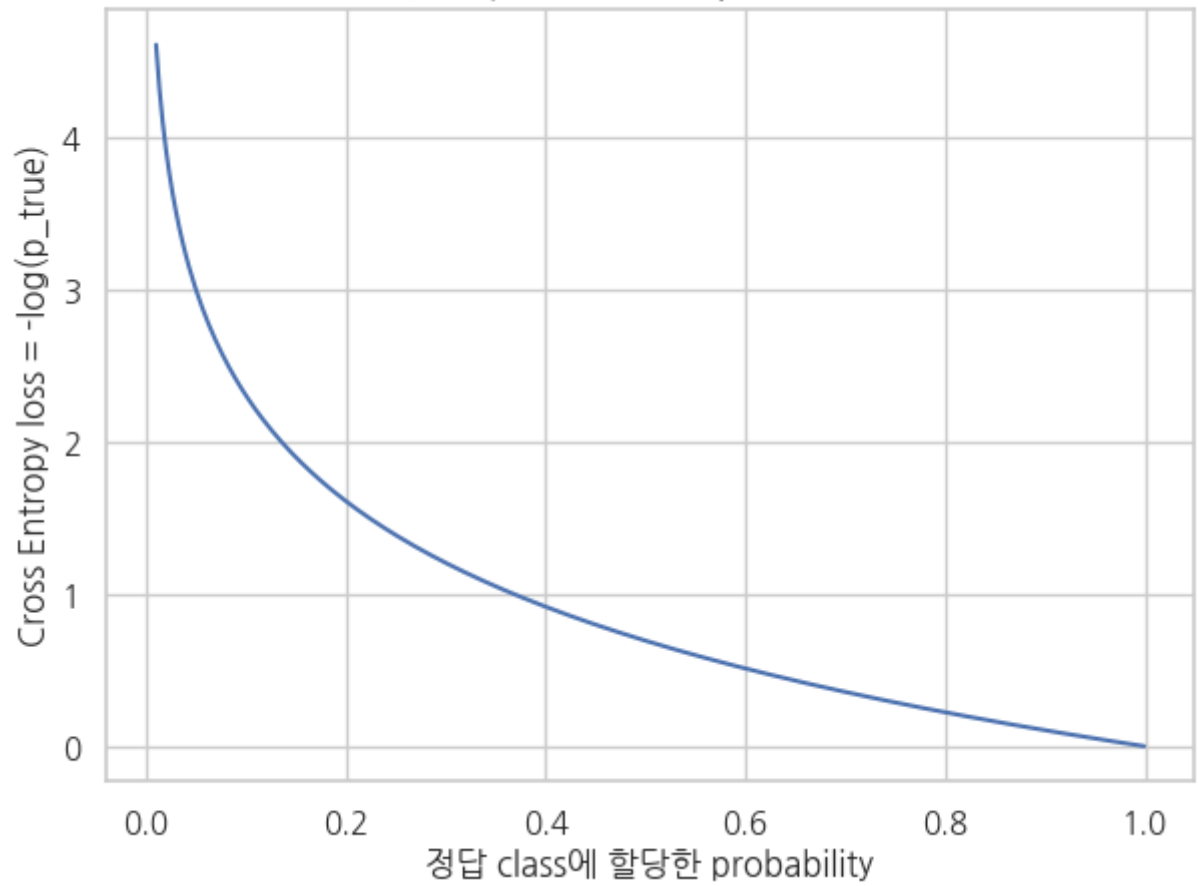
즉 정답 class에 준 확률만 직접적으로 읽으면 됨.

- 정답 확률 0.99 → loss 작음
- 정답 확률 0.50 → loss 중간
- 정답 확률 0.01 → loss 매우 큼

모델이 틀렸을 뿐 아니라 자신 있게 틀린 경우 더 큰 패널티를 줌.

```
# 21. 정답 확률과 CE loss 관계
pt = np.linspace(0.01, 0.999, 300)
ce = -np.log(pt)
plt.figure(figsize=(7,5))
sns.lineplot(x=pt, y=ce)
plt.xlabel("정답 class에 할당한 probability")
plt.ylabel("Cross Entropy loss = -log(p_true)")
plt.title("정답 확률이 높아질수록 Loss는 감소")
plt.show()
```


정답 확률이 높아질수록 Loss는 감소



Part 1 2 . 제공 실습 구조와 동일한 3-layer MLP 구현

제공 MLP 실습의 구조를 따름.

64
↓
128 + ReLU + Dropout
↓
64 + ReLU + Dropout
↓
10 logits

전체 구조를 수식으로

$$h_1 = \text{ReLU}(W_1x + b_1)$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2)$$

$$z = W_3h_2 + b_3$$

최종적으로:

$$f(x) = W_3\text{ReLU}(W_2\text{ReLU}(W_1x + b_1) + b_2) + b_3$$

이 식이 코드의 다음 구조와 대응함.

```
nn.Linear(64, 128)
nn.ReLU()
nn.Linear(128, 64)
nn.ReLU()
nn.Linear(64, 10)
```

```
# 22. Digits MLP
class DigitsMLP(nn.Module):
    def __init__(self, input_dim=64, num_classes=10, hidden_dims=(128,64), dropout=0.2):
        super().__init__()
        h1, h2 = hidden_dims
        self.net = nn.Sequential(
```

```

        nn.Linear(input_dim, h1),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(h1, h2),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(h2, num_classes)
    )

    def forward(self, x):
        return self.net(x)

digits_model = DigitsMLP().to(DEVICE)
print(digits_model)

n_params = sum(p.numel() for p in digits_model.parameters() if p.requires_grad)
print("\n학습 가능한 parameter 수:", n_params)

```

```

DigitsMLP(
  (net): Sequential(
    (0): Linear(in_features=64, out_features=128, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.2, inplace=False)
    (3): Linear(in_features=128, out_features=64, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.2, inplace=False)
    (6): Linear(in_features=64, out_features=10, bias=True)
  )
)

```

학습 가능한 parameter 수: 17226

파라미터 수를 직접 계산하기

첫 층 `Linear(64,128)`

Weight:

$$128 \times 64 = 8192$$

Bias:

$$128$$

합:

$$8320$$

두 번째 `Linear(128,64)`

$$64 \times 128 + 64 = 8256$$

출력층 `Linear(64,10)`

$$10 \times 64 + 10 = 650$$

전체:

$$8320 + 8256 + 650 = 17226$$

ReLU와 Dropout에는 학습되는 weight/bias가 없음.

```
# 23. 한 배치의 층을 통과하면서 shape가 어떻게 바뀌는지 확인
xb, yb = next(iter(dtrain_loader))
xb = xb.to(DEVICE)

layer_outputs = []
h = xb
for i, layer in enumerate(digits_model.net):
    h = layer(h)
    layer_outputs.append((i, layer.__class__.__name__, tuple(h.shape)))

pd.DataFrame(layer_outputs, columns=["index", "layer", "output shape"])
```

	index	layer	output shape
0	0	Linear	(6 4 , 1 2 8)
1	1	ReLU	(6 4 , 1 2 8)
2	2	Dropout	(6 4 , 1 2 8)
3	3	Linear	(6 4 , 6 4)
4	4	ReLU	(6 4 , 6 4)
5	5	Dropout	(6 4 , 6 4)
6	6	Linear	(6 4 , 1 0)

Part 1 3 . 실제 학습 – Adam + Early Stopping

제공 실습에서 사용한 핵심 요소:

- optimizer: Adam
- loss: CrossEntropyLoss
- validation 성능 관찰
- `model.train()` / `model.eval()`
- Early Stopping
- best checkpoint 개념

여기서는 HTML에서도 결과를 그대로 볼 수 있도록 메모리에서 best state를 보관함.

```
# 24. train/evaluate 함수
def train_multiclass_epoch(model, loader, optimizer):
    model.train()
    total_loss, total_correct, total = 0.0, 0, 0

    for xb, yb in loader:
```

```

xb, yb = xb.to(DEVICE), yb.to(DEVICE)

optimizer.zero_grad()
logits = model(xb)
loss = torch.nn.functional.cross_entropy(logits, yb)
loss.backward()
optimizer.step()

total_loss += loss.item() * len(xb)
total_correct += (logits.argmax(1) == yb).sum().item()
total += len(xb)

return total_loss/total, total_correct/total

def eval_multiclass(model, loader):
    model.eval()
    total_loss, total_correct, total = 0.0, 0, 0
    preds, targets = [], []

    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits = model(xb)
            loss = torch.nn.functional.cross_entropy(logits, yb)

            total_loss += loss.item() * len(xb)
            total_correct += (logits.argmax(1) == yb).sum().item()
            total += len(xb)
            preds.append(logits.argmax(1).cpu())
            targets.append(yb.cpu())

    return (
        total_loss/total,
        total_correct/total,
        torch.cat(preds).numpy(),
        torch.cat(targets).numpy()
    )

```

25. 학습 실행

```

digits_model = DigitsMLP().to(DEVICE)
optimizer = torch.optim.Adam(digits_model.parameters(), lr=1e-3, weight_decay=1e-4)

best_val_loss = float("inf")
best_state = None
patience = 6
no_improve = 0
history = []

```

```

for epoch in range(1, 61):
    train_loss, train_acc = train_multiclass_epoch(digits_model, dtrain_loader, optimizer)
    valid_loss, valid_acc, _, _ = eval_multiclass(digits_model, dvalid_loader)

    history.append({
        "epoch": epoch,
        "train_loss": train_loss,
        "valid_loss": valid_loss,
        "train_acc": train_acc,
        "valid_acc": valid_acc
    })

    if valid_loss < best_val_loss - 1e-5:
        best_val_loss = valid_loss
        best_state = {k: v.detach().cpu().clone() for k,v in digits_model.state_dict().items()}
        no_improve = 0
    else:
        no_improve += 1

    if no_improve >= patience:
        print("Early stopping at epoch", epoch)
        break

history = pd.DataFrame(history)
digits_model.load_state_dict(best_state)

test_loss, test_acc, test_pred, test_true = eval_multiclass(digits_model, dtest_loader)

print("Best validation loss:", best_val_loss)
print("Test loss:", test_loss)
print("Test accuracy:", test_acc)
display(history.tail())

```

```

Early stopping at epoch 47
Best validation loss: 0.0454250772173206
Test loss: 0.10378273005286852
Test accuracy: 0.9833333333333333

```

	epoch	train_loss	valid_loss	train_acc	valid_acc
4 2	4 3	0.0 1 2 0	0.0 5 0 7	0.9 9 7 2	0.9 8 3 3
4 3	4 4	0.0 1 1 9	0.0 5 4 2	0.9 9 9 3	0.9 7 7 8
4 4	4 5	0.0 0 8 0	0.0 5 1 0	0.9 9 8 6	0.9 8 3 3
4 5	4 6	0.0 1 5 6	0.0 5 6 2	0.9 9 6 5	0.9 7 7 8
4 6	4 7	0.0 1 2 9	0.0 5 6 4	0.9 9 5 8	0.9 7 7 8

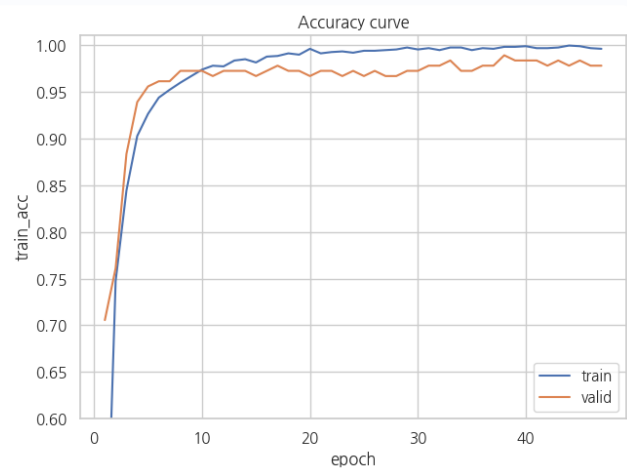
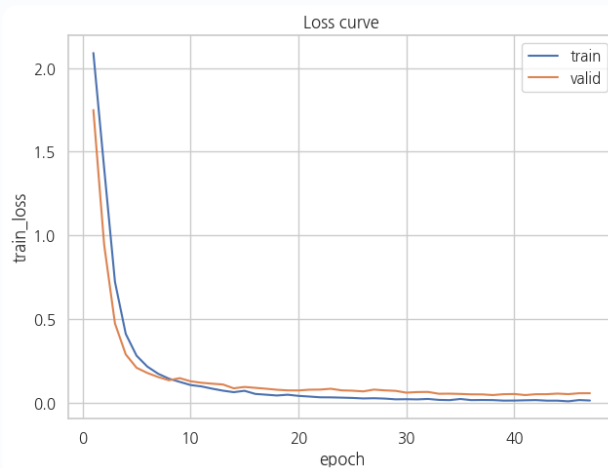
26. 학습 곡선

```
fig, axes = plt.subplots(1, 2, figsize=(13,5))
```

```
sns.lineplot(data=history, x="epoch", y="train_loss", label="train", ax=axes[0])
sns.lineplot(data=history, x="epoch", y="valid_loss", label="valid", ax=axes[0])
axes[0].set_title("Loss curve")
```

```
sns.lineplot(data=history, x="epoch", y="train_acc", label="train", ax=axes[1])
sns.lineplot(data=history, x="epoch", y="valid_acc", label="valid", ax=axes[1])
axes[1].set_title("Accuracy curve")
axes[1].set_ylim(0.6, 1.01)
```

```
plt.tight_layout()
plt.show()
```



학습 곡선에서 읽어야 하는 패턴

정상 학습

```
train loss ↓  
valid loss ↓
```

과적합 가능성

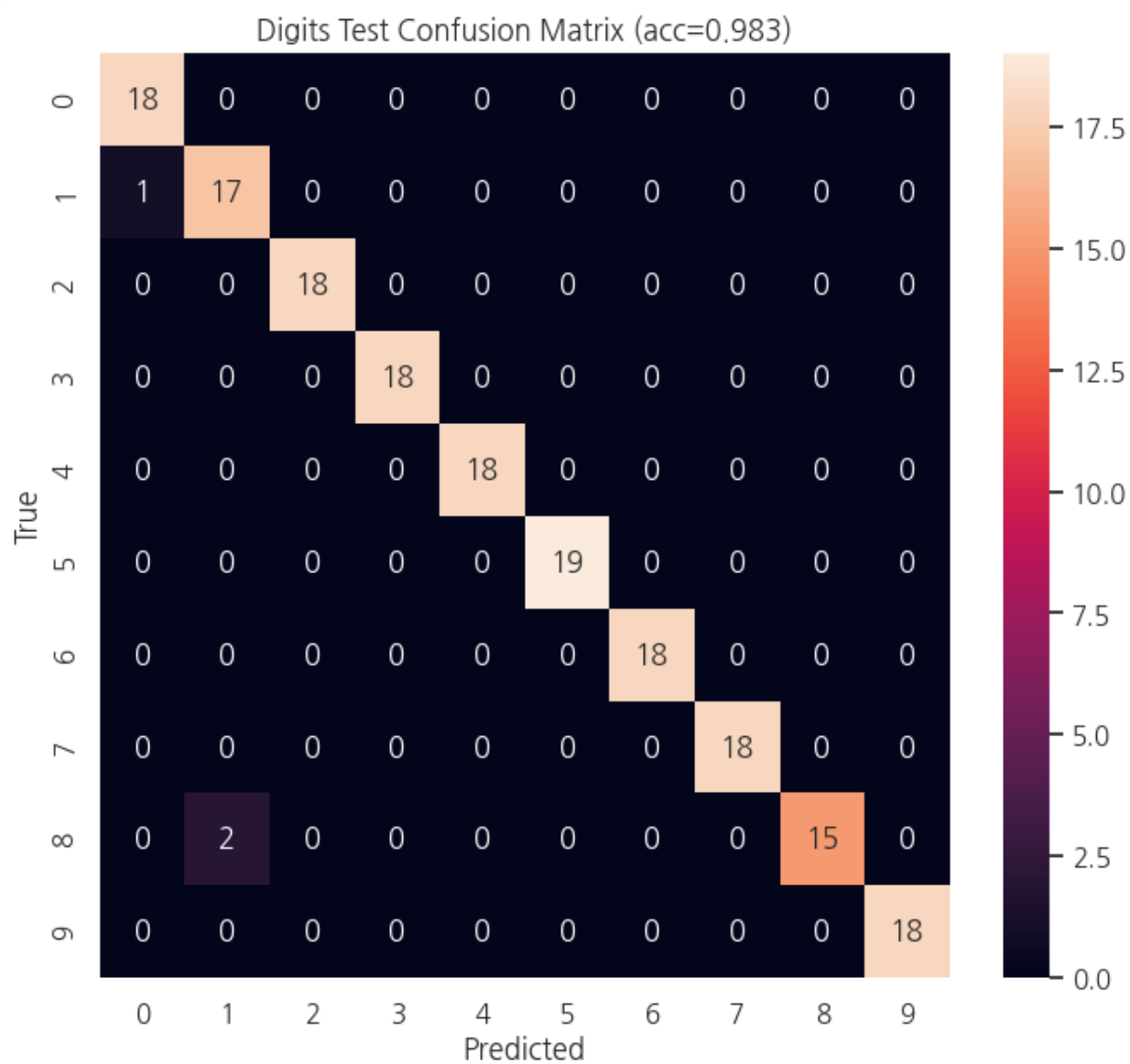
```
train loss 계속 ↓  
valid loss 다시 ↑
```

학습 데이터에는 더 잘 맞지만 새로운 데이터 일반화가 나빠지는 상태임.

Early Stopping은 validation loss가 일정 기간 개선되지 않으면 학습을 멈춰 과도한 학습을 방지하는 방법임.

Dropout, weight decay 역시 일반화에 도움을 주는 정규화 수단임.

```
# 27. Confusion Matrix  
cm = confusion_matrix(test_true, test_pred)  
  
plt.figure(figsize=(8,7))  
sns.heatmap(cm, annot=True, fmt="d")  
plt.xlabel("Predicted")  
plt.ylabel("True")  
plt.title(f"Digits Test Confusion Matrix (acc={test_acc:.3f})")  
plt.show()
```



Confusion Matrix 해석

행 = 실제 정답

열 = 모델 예측

대각선이 크면 정답을 많이 맞춘 것임.

예를 들어 (8행, 3열) 값이 크다면:

실제 숫자 8 을 모델이 3 으로 자주 오해

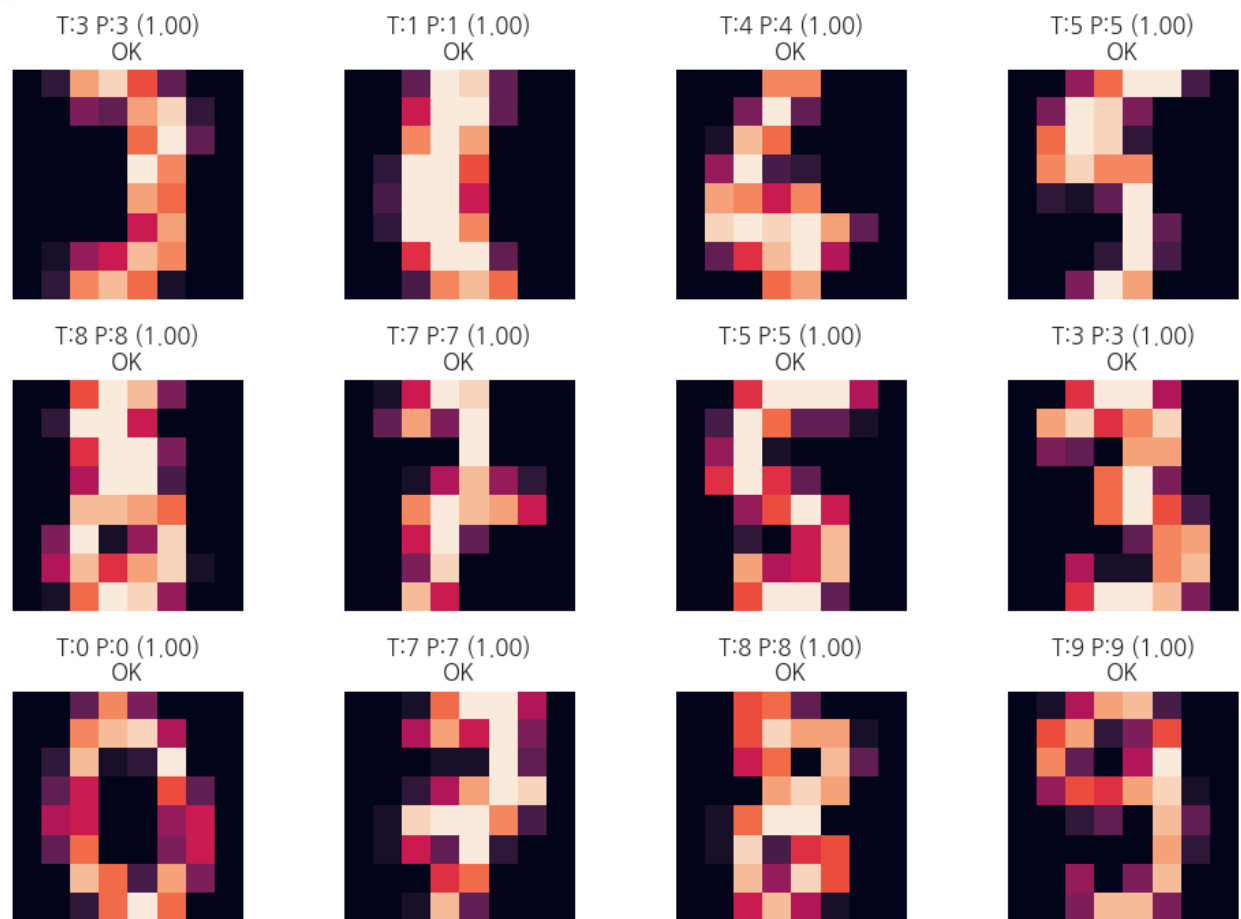
했다는 뜻임.

Accuracy 하나만으로는 알 수 없는 어떤 클래스끼리 헷갈리는가를 분석할 수 있음.

```
# 28. 샘플별 예측 확률 확인
digits_model.eval()
indices = np.arange(min(12, len(Xd_test_s)))
x_sample = torch.tensor(Xd_test_s[indices], dtype=torch.float32, device=DEVICE)

with torch.no_grad():
    logits = digits_model(x_sample)
    probs = torch.softmax(logits, dim=1).cpu().numpy()
    preds = probs.argmax(1)

fig, axes = plt.subplots(3, 4, figsize=(10, 7))
for i, ax in enumerate(axes.flat):
    ax.imshow(Xd_test[indices[i]].reshape(8,8))
    conf = probs[i, preds[i]]
    ok = preds[i] == yd_test[indices[i]]
    ax.set_title(f"T:{yd_test[indices[i]]} P:{preds[i]} ({conf:.2f})\n{'OK' if ok else 'WRONG'}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```



```
# 29. 한 샘플의 10개 class probability 막대그래프
i = 0
prob_one = pd.DataFrame({
    "digit": np.arange(10),
    "probability": probs[i]
})

plt.figure(figsize=(8,4))
sns.barplot(data=prob_one, x="digit", y="probability", hue="digit", legend=False)
plt.title(f"한 샘플의 class 확률 — true={yd_test[indices[i]]}, pred={preds[i]}")
plt.ylim(0,1)
plt.show()

display(prob_one.sort_values("probability", ascending=False).head())
```



	digit	probability
3	3	9 . 9 8 0 6 e- 0 1
2	2	1 . 7 2 7 2 e- 0 3
7	7	1 . 6 3 8 9 e- 0 4
9	9	3 . 8 4 5 9 e- 0 5
5	5	6 . 6 6 3 7 e- 0 6

Part 1 4 . MLP를 “계산 그래프”로 다시 요약

이진 분류

```
x
↓
Linear:  $z_1 = W_1x + b_1$ 
↓
ReLU:  $a_1 = \max(0, z_1)$ 
↓
Linear
↓
ReLU
↓
Linear
↓
logit
↓
BCEWithLogitsLoss
↓
loss
↓
backward
↓
각  $W, b$ 의 gradient
↓
Adam / SGD
↓
새  $W, b$ 
```

다중 분류

```
64 pixel features
↓
hidden representation
↓
10 logits
```

↓
CrossEntropyLoss
↓
gradient
↓
parameter update

Part 1 5 . 각 개념의 역할을 한 문장으로

개념	역할
Tensor	숫자 데이터를 PyTorch 연산 단위로 표현
Linear	$Wx + b$ 로 Feature를 새로운 표현으로 변환
Weight	입력 신호를 얼마나 강하게 반영할지 결정
Bias	뉴런의 활성화 기준을 이동
ReLU	비선형성을 추가하여 복잡한 패턴 표현 가능
Logit	확률 변환 전 모델의 원시 점수
Sigmoid	이진 logit을 0 ~ 1로 변환
Softmax	다중 class logits를 합이 1인 분포로 변환
Loss	현재 예측이 얼마나 나쁜지 수치화
Gradient	parameter를 조금 바꿀 때 loss가 변하는 방향/크기
Backpropagation	chain rule로 모든 parameter의 gradient 계산
Optimizer	gradient를 사용해 parameter 실제 업데이트
Batch	한 번의 update에 사용하는 샘플 묶음
Epoch	전체 training set을 한 번 사용
Validation	학습 중 일반화 상태를 관찰
Test	최종 모델의 새로운 데이터 성능 측정

개념	역할
Dropout	학습 중 일부 activation을 제거해 과적합 완화
Early Stopping	validation 개선이 멈추면 학습 종료

Part 1 6 . 지식 노드 연결

[선형회귀의 $Wx+b$]

↓ 같은 선형 계산

[Perceptron]

↓ 비선형 문제의 한계

[ReLU / 활성화 함수]

↓ 함수 중첩

[MLP]

↓ 예측과 정답 비교

[Loss]

↓ 미분

[Gradient]

↓ chain rule

[Backpropagation]

↓ 업데이트

[Optimizer]

↓ 반복

[Training Loop]

↓ 일반화 확인

[Validation / Test]

Part 1 7 . 자가 점검 문제

Q 1

`Linear(64, 128)`의 weight shape와 bias shape는?

Q 2

`Linear` → `Linear` 사이에 활성화 함수가 없으면 왜 깊게 쌓아도 표현력이 제한되는가?

Q 3

ReLU 입력값이 음수일 때 해당 지점의 derivative는 일반적으로 얼마로 처리되는가?

Q 4

이진 분류에서 logit이 0 이면 sigmoid 확률은?

Q 5

`optimizer.zero_grad()` 를 하지 않으면 무슨 일이 발생하는가?

Q 6

왜 validation/test 데이터에 `scaler.fit()` 을 하면 안 되는가?

Q 7

1 0 -class 분류 모델의 마지막 Linear 출력 차원은 왜 1 0 인가?

Q 8

`CrossEntropyLoss` 전에 모델에 Softmax를 붙이지 않는 이유는?

Q 9

`model.eval()` 이 특히 Dropout 모델에서 중요한 이유는?

Q 10

train loss는 계속 감소하지만 validation loss가 증가하기 시작하면 무엇을 의심해야 하는가?

정답

1. weight `(128,64)`, bias `(128,)`

2. 선형함수의 합성은 다시 선형함수로 합칠 수 있기 때문

3. 0

4. 0.5

5. 이전 batch의 gradient와 새 gradient가 누적됨

6. 미래 데이터 통계가 학습 과정에 들어가는 data leakage 발생

7. 각 class 0 ~ 9 에 대한 logit 하나씩 필요

8. `CrossEntropyLoss`가 raw logits를 받아 안정적인 log-softmax 계산을 내부 처리

9. train 모드에서는 Dropout이 랜덤하게 뉴런을 끄므로 추론 결과가 흔들릴 수 있음

0. 과적합

다음 학습 권장 순서

MLP



활성화 함수 비교: ReLU / Sigmoid / Tanh



Optimizer: SGD / Momentum / Adam



초기화: Xavier / He



Batch Normalization



CNN



Attention / Transformer